



Write.OnMars · Editor (app local-first)

⚙️ Status	In Progress
📊 Progress	0%
📁 Archive	☐
🏠 Area	● onMars

Documento de proyecto · Editor de escritura local-first (cara B) y core del editor de **Zeeds** (cara A, red social de lectura). Mantiene Spec Kit y git por debajo. Estado: diseño / pre-MVP. Última actualización: 2026-06-21.

1. Visión

Una aplicación de escritura **local-first**, de escritorio (y más adelante web), que funciona como un **cockpit editorial**: abre un proyecto, lee su contenido en Markdown, te deja editarlo y comentarlo, y orquesta agentes de IA para revisarlo — con git como única fuente de verdad. Es la **cara B** de **Zeeds**: mientras Zeeds es la red social de lectura/publicación (cara A), este editor es la herramienta de **autoría**, y a la vez el **core del editor de Zeeds**.

Es **agnóstica de género**: guías técnicas, novelas, guiones, artículos, poemas. Lo que cambia entre géneros son las *lentes de revisión*, el **tipo** de cada unidad y el vocabulario, configurables por proyecto; el editor, el git y el bucle de comentarios no cambian.

2. Zeeds: la relación cara A / cara B

Son dos mitades complementarias de **una sola plataforma**, con una moneda común: el **Markdown**.

- **Zeeds (cara A)** — lectura social, publicación, descubrimiento, monetización. Server (Next.js / Postgres / Vercel), público, el lado del **lector**. Las interacciones viven en base de datos, ancladas al texto publicado.
- **Editor (cara B)** — el cockpit de autoría local-first, asistido por IA. Git, privado por defecto, el lado del **autor**. Es el core del editor de Zeeds (su Fase 2 "Notion-lite" y su Fase 4 "Enterprise con IA de coherencia").
- **El puente = publicar**. Zeeds guarda el contenido como Markdown; el editor también. Publicar = empujar un Tree/Branch desde el repo local a un Forest de Zeeds. **Autoría local, lectura social**. El local-first hace que lo privado sea el estado nativo; publicar es lo que se añade encima (lo contrario del SaaS típico — argumento fuerte para enterprise).

Convergencias que validan la alianza: el asistente de IA de Zeeds (Fase 4) es "*de coherencia y redacción, no de escritura*" = nuestra revisión de consistencia (respeta "detector ≠ corrector"); el editor "Notion-lite Markdown" (Fase 2) = nuestro MVP; los **TAG** leafs de Zeeds ≈ nuestras lentes/comentarios.

3. Modelo de entidades — lenguaje ubicuo (botánico)

Cara A y cara B hablan el **mismo idioma**, una metáfora botánica completa:

Entidad	Qué es	En disco / sistema
Zeed	La idea germinal: el brief/spec del que nace el proyecto. Se planta.	brief (hoy spec.md), presentado como panel
Root	Recurso/fuente que nutre y <i>fundamenta</i> (grounding): story bible, citas, RAG.	roots/ (antes resources/)
Tree	El proyecto/obra que crece del Zeed.	el repo
Branch	Unidad de contenido, recursiva , con su tipo (capítulo, parte, escena, poema, post, artículo...).	content/ , ficheros .md
Leaf	Interacción anclada (comentario/reacción/tag). Del autor (revisión con prioridad) o del lector (social).	CriticMarkup inline (autor) / Postgres (lector)

Entidad	Qué es	En disco / sistema
Forest	Un conjunto de Trees: el espacio donde se publica, se descubre y se colabora. Público o privado (es una propiedad, no otra entidad).	la capa Zeeds

El pipeline se lee como un ciclo botánico: **plantas un Zeed** (brief) → **echa Roots** (research/recursos) → **toma forma de Tree** (plan = estructura de Branches) → **crecen las Branches** (escribir) → **se comprueba el enraizamiento** (claims/consistencia: cada Branch bien anclada en sus Roots) → **salen Leaves** (comentarios) → **se publica a un Forest**.

Notas: "capítulo" deja de ser carpeta y pasa a ser un **tipo** de Branch. La **carpeta** en disco sigue siendo **content/** (no **branches/**) para no colisionar con las *branches de git*. "Grounding" no es metáfora decorativa: fundamentar en fuentes se llama así en IA, y las Roots son literalmente eso. Renombres en contratos (bump de la feature 003): **capítulo** → **branch_id**, **by_chapter** → **by_branch**, **resources/** → **roots/**, brief = Zeed.

4. Estructura de producto y monetización

Decisión de trabajo: una plataforma integrada (marca Zeeds), con el tier en los Forests, construida "editor primero". Razonamiento (patrón GitHub):

- Equipo pequeño → una marca, una historia, un núcleo de código. Dos marcas dispersan.
- Núcleo compartido (motor del Tree) → no forkear en dos codebases.
- **Dos líneas de ingreso, un ecosistema.** El **editor local nunca se gatea** (anzuelo, gratis para siempre); lo que se gatea es la **IA / los modelos**.
- **Vivarium = freemium individual, palanca en los LLMs.** Free → **BYOM** (traes tus claves/modelos locales y pagas tú al proveedor; ya es principio del framework). Pro → **modelos gestionados por nosotros** (hosted, sin configurar) + IA premium (pipeline de revisión completo, más agentes/lentes, límites). Playbook Cursor / Zed.
- **Zeeds = negocio de plataforma / red.** Forests privados, colaboración, integraciones/webhooks y monetización de creadores (Substack/Twitch). Aquí está el dinero de equipo / enterprise.

- **Mapeo GitHub:** Zeeds ≈ GitHub (plataforma/orgs, dinero enterprise); Vivarium ≈ Copilot (add-on de IA por asiento, freemium). El **enterprise compra el bundle**: Forests privados (Zeeds) + Vivarium Pro por asiento (modelos hosted, admin/SSO).
- **Orden de construcción: editor primero** (lo más difícil y diferenciado, ya especificado), y la capa Forest/social encima cuando haya con qué llenarla.

Free tier: un **gran Forest público** al inicio (concentrar la actividad; el propio MVP de Zeeds lanza con "un solo lugar"), y Forests por género/tipo **más tarde**, al haber masa crítica (como Reddit → subreddits).

Nombre del editor: Vivarium (apodo corto: **Viv**). Latín = 'lugar de vida': entorno controlado donde un Zeed se cultiva hasta volverse Tree antes de salir al Forest. Elegido por alcance internacional. Zeeds = la plataforma y los Forests; Vivarium = el editor.

5. Estructura de carpetas (estrategia "fachada")

Regla fijada: solo tocamos el proceso hasta donde Spec Kit nos deje. El editor presenta una estructura lógica limpia, desacoplada de la física: solo `content/` se mueve de verdad; el proceso se queda donde Spec Kit lo quiere y se muestra por paneles.

```

mi-tree/
├─ content/                # CONTENIDO – árbol recursivo d
e Branches (nuestro, libre)
|   └─ 01-introduccion.md  # Branch hoja · tipo: capít
tulo
|   └─ 02-fundamentos/     # Branch contenedor · tip
o: parte
|       └─ 01-embeddings.md
|       └─ 02-atencion.md
├─ roots/                  # ROOTS – fuentes y story bible
(antes resources/)
|   └─ sources/             # fuentes / citas
|   └─ bible/              # ficción: personajes, lug
ares, reglas

```

```

├─ exports/                                # SALIDAS – PDF y otros
├─ specs/<###-feature>/                  # PROCESO (backend Spec Kit) →
el editor lo muestra como PANELES:
|   └─ spec.md                          → panel "Zeed (brief)"
|   └─ plan.md                          → panel "Outline / estructura del Tre
e"
|   └─ research.md · findings.md · claims.md · glossary.md
→ paneles
├─ .specify/                             # ancla de Spec Kit – NO se toc
a
└─ .writeonmars-manifest.json

```

Verificado con subagente contra el código real de Spec Kit: `chapters/` → `content/` es 100% nuestro y no rompe nada; `specs/` y los nombres `spec.md/plan.md` están cableados en el core (mover la carpeta se puede con `feature_directory`, pero renombrar archivos core acopla a internals → evitar); `.specify/` es el ancla, no se toca.

6. Cómo se apoya en la arquitectura existente

Pieza del frontal	Se apoya en
Stepper del pipeline	<code>status.py --json</code> (<code>next_step</code> , <code>gates</code> , <code>by_branch</code> , <code>all_chapters_approved</code>)
Tablero de Branches (kanban)	estados por Branch (drafting / in_review / needs_revise / approved)
Panel de Roots / claims inline	<code>research.md</code> (CitationRecord) + <code>claims.md</code> (feature 003)
Editor + Leaves (comentarios)	<code>content/*.md</code> • CriticMarkup; comentarios → <code>feedback_intake.py</code> → revise
Nuevo proyecto / scaffold	<code>tools/new-guide.sh</code>
Zeed + adendas guiados	<code>speckit.specify</code> (brief) + <code>speckit.constitution</code>
Historial / ramas / colaboración	git (worktrees por Branch; Forest = remoto)
Roots / RAG	<code>roots/</code> • <code>index.py</code> • RAG externo vía MCP

Pieza del frontal	Se apoya en
Exportar	<code>export.py</code> (a demanda)

7. MVP (v1): editor + comentarios, sin orquestación

- Abrir un proyecto (Tree) y ver el árbol de Branches.
- Editar en un **editor visual estilo Notion** (WYSIWYG por bloques), simplificado a: **títulos, párrafos, negrita, cursiva, subrayado y enlaces**. Por debajo, Markdown en git.
- Dejar **Leaves (comentarios CriticMarkup) con prioridad**.
- **Modo foco**.
- **Exportar PDF a demanda** (manual; nada automático).
- Historial git visible.

Nota del editor: WYSIWYG por bloques, pero almacena Markdown. Negrita/cursiva/enlaces serializan directo; el subrayado (sin Markdown nativo) como `<u>` o convención del proyecto. CriticMarkup va por encima.

Cambio de flujo: de "auto → export" a "agentes terminan → `in_review` → el humano edita y comenta → export manual". Ajustes: desacoplar export de close; `feedback_intake.py` lee CriticMarkup del `.md` (regex) en vez de PDF; la Editora jefa no auto-avanza las etapas globales.

8. Modelo de comentarios (Leaves del autor)

Dos ejes separados a propósito:

- **Intención** → texto libre.
- **Prioridad** → lo único que solo tú sabes. Reutiliza la severidad del pipeline: **crítico / medio / bajo**. Crítico bloquea; bajo avisa. Enchufa en la feature 003.

El **routing a quién revisa lo decide el agente** (triaje, transparente), no el humano. Override manual opcional. Resolución = el comentario **desaparece** al hacer commit (git es el historial). Formato CriticMarkup `{>> ... <<}`, encontrable por regex.

9. Configuración por proyecto + plantillas

Vocabulario configurable por proyecto (adenda-archivo en el repo, como las adendas de la constitución): niveles de prioridad (nombres/colores), las **lentes** del triaje (no-ficción: precisión/fuentes; novela: continuidad/arco/voz), y **plantillas** (guía/novela/guion) de las que arrancas y modificas. El panel de Roots se vuelve *story bible* en ficción.

10. Etapa 2 — Inyectar el proceso Write.OnMars

- **Nuevo proyecto desde la app:** asistente que ejecuta por detrás `tools/new-guide.sh` (repo + `specify init` + `preset add` + `bootstrap.py` + commit). Sin terminal.
- **Captura guiada del Zeed + constitución:** la app pregunta y rellena el brief de 9 campos (`speckit.specify`) y las adendas sector+tono+terminología (`speckit.constitution`); visibles y editables.
- **RAG externo (Roots):** conectar un RAG externo como base de fuentes, vía MCP compatible con el contrato de citación.
- **Revisión de consistencia contra los Roots:** pides a un agente que revise una Branch contra los Roots del proyecto y señale inconsistencias. **Generaliza la maquinaria de claims (003):** la "fuente" puede ser un Root (ficha de personaje, regla del mundo, fuente académica) y la relación (apoya/matiza/contradice/menciona) se vuelve el veredicto de consistencia. Ficción: "esto va en contra del arco del personaje". Académico: afirmaciones que contradicen las fuentes. El resultado = Leaves con prioridad sobre el texto.

11. Colaboración y local-first (la capa Forest)

El local-first **no pierde** la colaboración: git ya resolvió la colaboración distribuida. El **Forest es el remoto compartido** (como [GitHub.com](https://github.com) es el remoto de git): cada persona tiene su Tree en local, colabora por sync/ramas/PRs contra el Forest, y sigue siendo local-first. El descubrimiento/investigación ocurre en el Forest (búsqueda entre Trees, Leaves sociales, TAG semánticos). La colaboración **en**

tiempo real (Google-Docs / CRDT, Fase 4 de Zeeds) es una capa posterior; la **asíncrona** (git/PR) la tienes desde el principio.

Bucle del ecosistema: un Tree publicado en el Forest puede convertirse en **Root** (fuente citable) del Tree de otra persona — el ángulo "fuente citable / OEmbed" del PDF de Zeeds.

12. El anclaje (PENDIENTE — el Core de ambos proyectos)

Ambos lo llaman "el Core" y es donde cara A y cara B divergen: el autor ancla Leaves **inline en el Markdown** (CriticMarkup, en git, versionado — ideal para autoría); Zeeds ancla Leafs **en Postgres** por prefix/suffix (v1) o `paragraph-id + offset` (v2, robusto a ediciones — ideal para lectura social a escala).

Probablemente compartan el **algoritmo** (paragraph-id + offset) y el Markdown como moneda, con **almacenamiento distinto** por audiencia. **A resolver en la siguiente sesión.**

13. Arquitectura técnica

- **Desktop-first, local-first.** Separar **(a) paquete de UI** agnóstico de plataforma y **(b) adaptador de plataforma** (fs, git, scripts, agentes) con impl. desktop y web. Así la UI viaja a web y solo se reescribe el adaptador.
- **Stack: Tauri** (decidido). Backend en Rust, ligero y afín a local-first; la UI (webview) se reutiliza para la versión web vía el adaptador de plataforma. Electron descartado (mismo reuso de UI, app más pesada).
- **Editor:** WYSIWYG por bloques (TipTap/ProseMirror o CodeMirror 6) que serializa a Markdown + capa CriticMarkup.
- **Git:** libgit2 / isomorphic-git. **Scripts** (`status.py` , `export.py` , `feedback_intake.py` , `index.py` , `new-guide.sh`) como **sidecar local**. **Agentes y RAG** vía MCP. **Forest/sync** = remoto git.
- **Zeeds (cara A):** Next.js + Vercel Postgres + NextAuth (del PDF). Cambio mínimo para el Branch recursivo: añadir `parent_branch_id` opcional (conservando `order`).

14. Dataset de correcciones y fine-tuning (el flywheel)

Vivarium recoge las decisiones del autor sobre lo que hace la IA para construir un **dataset de entrenamiento**, con el que fine-tunear un modelo que escriba mejor prosa. Ese modelo se convierte en el **Vivarium Pro** hosted (cierra el loop: free=BYOM / Pro=nuestro modelo). El foso competitivo lo construyen los usuarios corrigiendo.

Señales que ya genera el editor (sin esfuerzo extra):

- **Pares de edición** (oro): borrador IA → versión final del autor. Supervised fine-tuning ("reescribe a calidad de casa"). El diff ya vive en git.
- **Accept/reject por hunk** (modelo Cursor): preferencia binaria → DPO/RLHF (chosen vs rejected).
- **Corrección dirigida por comentario**: (texto, comentario/crítica, corrección) → instruction-tuning.
- **Aceptado tal cual**: también es señal positiva (el modelo acertó), no solo las correcciones.

Lo que hay que añadir: un **log de procedencia/decisiones** (p. ej.

`.vivarium/decisions.jsonl`) que etiquete qué vino de la IA, la disposición del humano (aceptar/rechazar/editar-a-esto) y el span anclado. Git da el texto; el log da las etiquetas. Cada corrección va anclada → mismo primitivo que el anclaje (otra razón por la que el anclaje es foundational).

Sinergias: las lentes/prioridad **etiquetan el dataset solas** (`@voz` = prosa/voz, el oro para este objetivo; `@precision` = factualidad). El modelo resultante es "**marcela-prose aprendida**", no escrita a mano con reglas.

Calidad: filtrar por tipo/lente — no todo edit es señal de prosa (typos, reorganización, datos).

Consentimiento — decidido: por fases. Fase 1 — solo tu data (local y privada): construyes un modelo de *tu* voz, sin infraestructura de consentimiento ni legal. Captura trivial. **Fase 2 — opt-in compartido** (más adelante): el usuario puede optar a contribuir sus correcciones al modelo compartido; requiere **consentimiento explícito, ToS/licencia y anonimización**. El local-first hace que contribuir sea siempre opcional, nunca por defecto.

15. Registro de decisiones

Decisión	Resultado
Proceso vs Spec Kit	Solo tocar hasta donde Spec Kit deje (fachada). <code>content/</code> se mueve; <code>specs/</code> • <code>.specify/</code> quietos, presentados por paneles. No renombrar archivos core ni mover <code>.specify/</code>
Modelo de entidades	Botánico: Zeed / Root / Tree / Branch (recursivo) / Leaf / Forest
Carpeta de contenido	<code>content/</code> en disco (no <code>branches/</code> , por colisión con git); "Branch" es la palabra del dominio/UI
Recursos	<code>resources/</code> → <code>roots/</code>
Relación con Zeeds	Cara A (social/Forests) ↔ cara B (editor local). Markdown = moneda. Publicar = empujar a un Forest
Estructura de producto	Una plataforma integrada (marca Zeeds), tier por Forests, construida "editor primero"
Monetización	Editor local gratis (anzuelo); Forests privados/enterprise + integraciones + monetización de creadores
Nombre del editor	Tentativo: Greenhouse/Invernadero (NO fijado, pendiente)
Free tier	Un gran Forest público al inicio; Forests por género/tipo más tarde
Colaboración	Forest = remoto git (async, preserva local-first); tiempo real (CRDT) más tarde
Editor MVP	Visual estilo Notion, simplificado: títulos, párrafos, negrita, cursiva, subrayado, enlaces. Almacena Markdown
Comentario (Leaf)	Prioridad crítico/medio/bajo; routing por triaje del agente; override opcional; se resuelve vía commit
Export	Manual, desacoplado de close
Plataforma técnica	Desktop-first local-first; UI reutilizable en web vía adaptador; Electron vs Tauri abierto
Anclaje	PENDIENTE (el Core)

16. Preguntas abiertas / siguientes pasos

- **Resolver el anclaje** (inline git vs paragraph-id+offset; algoritmo compartido).
- (Resuelto) Nombre del editor: **Vivarium** (apodo corto: Viv).
- Editor WYSIWYG base (TipTap vs CodeMirror) y serialización del subrayado.
- RAG externo a integrar.
- Forma del free tier (un Forest vs por género) — diferida.
- Repositorio aparte para el editor (no vive en `writing-framework`).

17. Fundamento (investigación de métodos en producción)

El modelo de Leaves + claims + revisión de consistencia se apoya en el patrón de producción para long-form con citas (**research** → **grounding** → **write** → **verify**): atribución a nivel de afirmación (Anthropic Citations API, "Attribute First, then Generate"), factualidad por afirmaciones soportadas/verificables (FactScore, VeriScore), y veredicto de relación de la cita — apoya/matiza/contradice/menciona — (Scite). Escritor ≠ revisor cross-model y spec-driven sobre git completan el cuadro. Detalle en la feature `003-atribucion-factualidad` del repo `writing-framework`.